Dense-to-Expert-MoE Upcycling: Imperative Baseline vs Declarative BrainSurgery

Same conversion semantics, but explicit checkpoint surgery and validation replace handwritten control flow.

A. Reference Python Script

```
moe_to_dense_mapping = {
  "feed_forward_moe.experts.mlp.w1": ...
  "feed_forward_moe.experts.mlp.w2": ...
  "attention.w_q.weight": ...
}

for expert, path in enumerate(dense_paths):
  dense_state_dict = load_state_dict(path)
  for key in list(moe_state_dict.keys()):
    if any(pattern in key for pattern in ...):
      dense_key = key.replace(...)
      if "expert" in key or "router" in key:
        moe_state_dict[key][...] =
          dense_state_dict[dense_key].T
      ...
```

Control flow, mapping, mutation, and output writing are intertwined.

B. BrainSurgery YAML Plan

```
transforms:
  - assert: { exists: m0::model.embed_tokens.weight }
  - assert:
      equal:
        left:  m0::model.layers\.(\d+)\.mlp...
        right: m1::model.layers.\1.mlp...
  - copy:
      from: m0::model.layers\.(\d+)\.mlp...
      to:   m0::model.layers.\1.mlp.experts.0...
  - copy:
      from: m1::model.layers\.(\d+)\.mlp...
      to:   m0::model.layers.\1.mlp.experts.1...
  - fill:
      to: m0::model.layers.\1.mlp.gate.weight
      mode: constant
      value: 0
  - delete: { target: m0::model.layers\.(\d+)\.mlp... }
```

Assertions, surgery, and validation are explicit, reviewable, and reusable.

C. Built-In Validation

```
inputs:
  - yaml::olmo_1b_0724_hf_dense_moe_demo
  - ref::olmo_1b_0724_hf_dense_moe_reference

transforms:
  - diff: { mode: aliases, left_alias: ref,
            right_alias: yaml }

Missing on left:
  (none)
Missing on right:
  (none)
Differing:
  (none)
No differences found.
```

The declarative plan matches the independent reference implementation.

Dense checkpoint A + Dense checkpoint B -> conversion -> MoE-style checkpoint -> BrainSurgery diff